AI Education

Agent tuteur testé & CI/CD

AI Education

Tests automatisés & CI/CD agentique

Mini-projet — KAMAL EL AFOUI · Jour 22

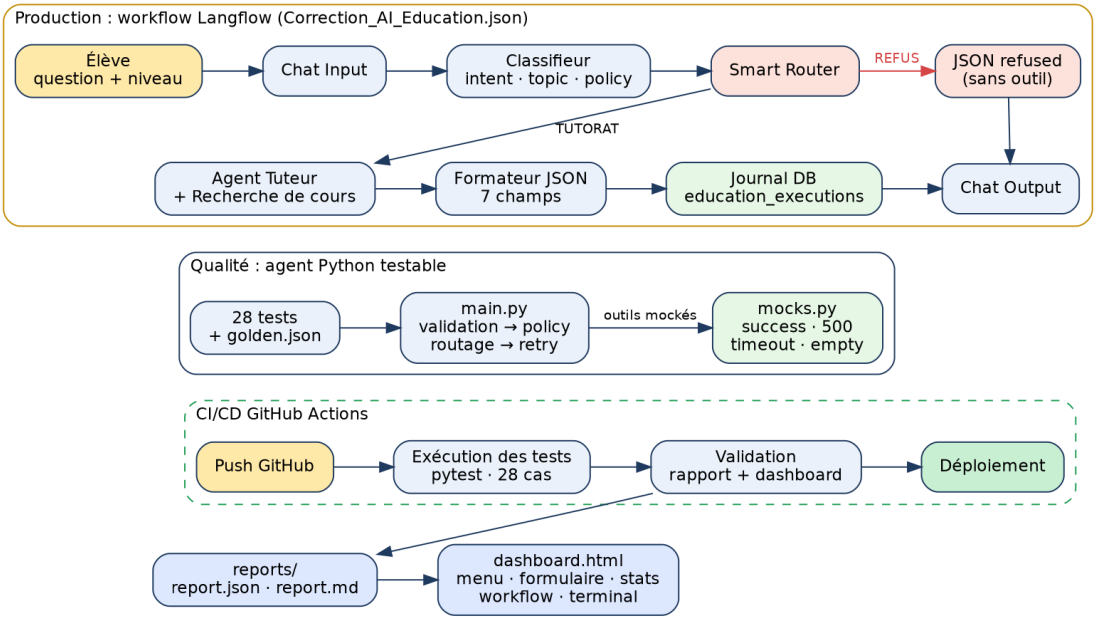
Version 2026-10-28

1. Objectif

Industrialiser un agent tuteur « AI Education » afin de garantir sa qualité après chaque modification : tests métier, mocks des outils, suite de non-régression et pipeline GitHub Actions.

Principe directeur : un comportement validé hier doit rester valide demain.

2. Architecture



En production, le workflow Langflow classe la demande, applique la policy, puis passe par l'agent tuteur et le formatage JSON. En qualité, l'agent Python (même logique) est testé avec des mocks (28 tests), exécutés à chaque push par la CI GitHub Actions, qui produit le rapport et le dashboard.

Composant	Rôle
main.py	Agent : validation, policy, routage, retry, réponse JSON avec trace
mocks.py	Mocks API Cours et API Progression (4 scénarios)
tests/test_agent.py	28 tests métier, compatibles unittest et pytest
tests/golden.json	Cas de référence pour la non-régression
run_report.py	Exécute les tests, produit report.json, report.md et dashboard.html
Correction_AI_Education.json	Export du workflow Langflow (9 composants, importable dans Langflow)
notebook.ipynb	Démonstration pas à pas
.github/workflows/ci.yml	Pipeline CI : tests, validation, déploiement

Workflow Langflow

Le fichier Correction_AI_Education.json suit le même schéma que l'agent Météo du cours : classification, routage, agent avec outil, formatage JSON, journalisation.

Composant	Rôle
Chat Input (Élève)	Reçoit la question de l'élève
OpenRouter (Classifieur)	Modèle du routeur ; prompt de classification : intent, topic, policy_flag, valid_input
Smart Router	2 routes : TUTORAT, ou REFUS / ENTRÉE INVALIDE / HORS CADRE (policy)
Chat Output (refus)	Renvoie directement un JSON status = refused, sans appel d'outil
Recherche de cours	Outil de l'agent (en test, remplacé par un mock)
Agent Tuteur	Explique, crée un quiz, retry ×1 puis service indisponible
OpenRouter (Formateur JSON)	Produit les 7 champs du contrat de sortie
Journal des exécutions (DB)	Insère le JSON dans la collection education_executions
Chat Output	Réponse finale

Avant l'import : renseigner dans Langflow la variable Openrouter, la clé du modèle de l'agent et l'URI de la base. Aucun identifiant n'est stocké dans le fichier.

3. Contrat de sortie (format)

Chaque réponse est un objet JSON avec les champs obligatoires suivants :

Champ	Type	Description
status	string	ok · refused · error · degraded
intent	string	explain · quiz · progress · policy · invalid
answer	string	Texte destiné à l'élève
level	string	primaire · college · lycee · universite
quiz	liste	Questions : q, choices, answer_index
sources	liste	Identifiants de cours ou d'élève utilisés
trace	liste	Appels d'outils : tool, attempt, ok, error

4. Policy (règles métier)

- Refus de toute aide à la triche (réponses d'examen).
- Refus des tentatives d'injection de prompt (« ignore tes instructions »).
- Refus des demandes hors cadre éducatif.
- Confidentialité : un élève ne peut pas consulter la progression d'un autre.
- Les refus ne déclenchent aucun appel d'outil. La détection ignore casse et accents.

5. Documentation des mocks

Certains scénarios sont rares ou difficiles à reproduire avec des services réels. Les mocks simulent succès, pannes et comportements inattendus sans réseau, de façon rapide et déterministe.

Scénario	API Cours	API Progression	Comportement attendu de l'agent
success	Retourne le cours	Retourne la progression	status = ok
error_500	Lève ToolError	Lève ToolError	Retry x1 puis status = degraded
timeout	Lève ToolTimeout	Lève ToolTimeout	Retry x1 puis status = degraded
empty	Retourne None	Retourne None	status = ok, message « Aucun cours »
FlakyCourseAPI	Échoue au 1er appel	—	Le retry réussit : status = ok, 2 appels tracés

Données simulées : 5 cours (fractions, équations, Python, photosynthèse, grammaire), une banque de quiz et 3 élèves (S001, S002, S003). Chaque mock compte ses appels (attribut calls) pour vérifier les enchaînements.

6. Suite de tests

28 tests au total (exigence : ≥ 20), répartis en 6 catégories :

Catégorie	Tests	Réussis	Objectif
Format	5	5	Schéma JSON, champs obligatoires, types
Intégration	5	5	Enchaînement agent → outils, trace
Policy	6	6	Triche, injection, confidentialité
Mauvaise entrée	6	6	Vide, None, niveau/ID invalides, trop long
Mocks	5	5	Panne 500, timeout, retry, vide
Non-régression	1	1	3 cas de référence (golden.json)

Liste détaillée

N°	Test	Catégorie
01	schema explain	Format
02	schema quiz	Format
03	quiz answer index valid	Format
04	output is json serializable	Format
05	validator detects missing field	Format
06	explain calls course api	Intégration
07	progress calls progress api only	Intégration
08	trace records tool call	Intégration

N°	Test	Catégorie
09	unknown topic graceful	Intégration
10	new student no progress	Intégration
11	refuse cheating	Policy
12	refuse prompt injection	Policy
13	refuse off topic	Policy
14	refuse other student progress	Policy
15	no tool call when refused	Policy
16	case and accents do not bypass policy	Policy
17	empty question	Mauvaise entrée
18	none request	Mauvaise entrée
19	invalid level	Mauvaise entrée
20	invalid student id	Mauvaise entrée
21	question too long	Mauvaise entrée
22	wrong type question	Mauvaise entrée
23	error 500 degraded	Mocks
24	timeout retries once	Mocks
25	retry recovers	Mocks
26	empty scenario	Mocks
27	progress outage degraded	Mocks
28	golden cases	Non-régression

7. Pipeline CI/CD

Étape	Action
Push sur GitHub	Déclenche le workflow à chaque push et pull request
Exécution des tests	pytest tests -v (28 cas)
Validation (build)	run_report.py : rapport, dashboard ; échec si un test est rouge
Déploiement	Job conditionné au succès des tests et à la branche main (simulé)

Pour obtenir le badge vert : créer le dépôt, pousser le code, puis remplacer <votre-compte> dans le badge du README.

8. Dashboard (dashboard.html)

Fichier HTML unique, sans dépendance, qui suit le mode clair ou sombre. Il est régénéré par run_report.py à chaque exécution des tests.

Page du menu	Contenu
Vue d'ensemble	Indicateurs (tests, réussis, échoués, taux), état du pipeline, anneau de réussite, tests par catégorie
Tester l'agent	Formulaire : question, élève, niveau, scénario de mock, case « progression d'un autre élève » ; affiche statut, réponse, quiz et JSON avec trace
Statistiques	Requêtes par statut (graphique en barres), durée des tests, historique des requêtes de la session
Tests	Tableau des 28 tests avec filtre par catégorie
Mocks	Les 4 scénarios simulés et la réaction attendue
Workflow	Schéma du workflow Langflow (Correction_AI_Education.json) et liste de ses 9 composants
Terminal	Simulation des commandes du projet dans le navigateur (voir section 9)

Le formulaire exécute une copie JavaScript de la logique de main.py. Si l'agent Python est modifié, cette copie doit être mise à jour.

9. Utilisation en terminal

Prérequis : Python 3.10 ou plus récent et Git. Sous Windows, remplacer python3 par python ou py.

Commande	Effet
python main.py "Explique-moi les fractions"	Exécute l'agent (options : --student, --level, --scenario)
python main.py "Donne un quiz python" --scenario timeout	Simule une panne : retry puis status degraded
python -m unittest discover -s tests -v	Lance les 28 tests sans rien installer
pytest tests -v	Lance les tests comme la CI (pip install pytest)
python run_report.py	Tests + reports/ + dashboard.html
git init, git add ., git commit, git push	Publie sur GitHub et déclenche le pipeline CI

10. Résultats

28/28 tests réussis (100 %) — rapport généré le 2026-10-28 11:26. Le détail visuel est disponible dans dashboard.html (section 8).

11. Livrables

- Workflow final : Correction_AI_Education.json (export Langflow) et cette documentation.
- Suite de tests (28 cas) avec rapport : tests/, reports/report.json, reports/report.md.
- Pipeline CI fonctionnel : .github/workflows/ci.yml (badge vert après premier push).
- Documentation des mocks : section 5.
- Code et démonstration : main.py, mocks.py, notebook.ipynb, présentation PPTX.
- Dashboard : dashboard.html (menu, formulaire, graphiques, terminal).
- Identité visuelle : logo (SVG et PNG) dans assets/.